Word Level Plagiarism Checker

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Varshini-2520080013
Saranya-2520080012
Manya - 2520090032

Team Number:16

Section: 13

Under the guidance of

Dr. Dyuti Banerjee

<Assistant Professor, Department of AI/DS>



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The Mini Project Report entitled "**Word level Plagiarism checker"** is a record of Bonafide work of **Varshini-2520080013, Saranya- 2520080012, Manya-2520090032** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering/Computer Science and Information Technology/ Artificial Intelligence & data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Varshini-2520080013

Saranya-2520080012

Manya- 2520090032

K L (Deemed to be) University

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATASCIENCE/ CSIT



CERTIFICATE

This is to certify that the mini project based report entitled "**Word Level plagiarism detector"** is a bonafide work done and submitted by **Varshini-2520080013, Saranya- 2520080012, Manya- 2520090032.** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering/ Computer Science and Information Technology/Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, Dr. Dyuti Banerjee, Assistant Professor, Department of AI/DS which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)/Dr. Khaja Shareef (CSIT)/ Dr. Sreenath Itikela (AI&DS)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Varshini-2520080013

Saranya-2520080012

Manya- 2520090032

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	14
8.	Conclusion & Future Scope	16
9.	References	17

1.ABSTRACT

Plagiarism detection has become increasingly important in educational institutions and research organizations due to the widespread availability of digital content. Manually comparing documents is time-consuming and inefficient, particularly when multiple research papers need to be analyzed. Existing plagiarism detection tools, such as Turnitin and iThenticate, provide similarity analysis but are often commercial, and their internal algorithms are not fully accessible for academic study.

The proposed Multi-Document Word-Level Plagiarism Detection and Similarity Analysis System aims to provide an efficient and educational approach to document comparison using Data Structures and Algorithms (DSA). The system allows users to upload multiple documents in PDF, DOCX, and TXT formats, extracts and preprocesses their text, and identifies matching words and phrases. It implements algorithms and data structures such as Knuth-Morris-Pratt (KMP), Rabin-Karp, Hash Maps, Inverted Indexing, and Dynamic Programming using the Longest Common Subsequence (LCS). Similarity measures, including Jaccard Similarity and Cosine Similarity, are used to calculate document similarity scores.

The system highlights matching content, ranks document pairs according to their similarity scores, and generates reports with configurable warning thresholds. It also incorporates a privacy-focused design in which unpublished documents are processed without being added to the permanent comparison repository by default. The proposed system demonstrates the practical application of advanced algorithms while providing a useful tool for identifying textual similarities and supporting academic integrity.

Team Name: Word level plagiarism checker

Team Members:

1. [Varshini, 2520080013]
2. [Saranya, 2520080012]
3. [Manya, 2520090032]

2. INTRODUCTION

In the modern digital era, the availability of online information, research publications, and academic documents has increased significantly. Although this makes knowledge more accessible, it also creates challenges in maintaining originality and academic integrity. Plagiarism, which involves using another person's words or ideas without appropriate acknowledgment, is a major concern in educational institutions and research organizations. Identifying similarities between documents is therefore an important step in reviewing academic work and encouraging proper citation practices.

Traditional manual document comparison becomes difficult and time-consuming when dealing with multiple research papers or lengthy documents. Existing plagiarism detection tools provide automated similarity analysis, but their internal implementations may not be accessible for academic learning, and some services require paid subscriptions. This creates an opportunity to develop an educational system that demonstrates how efficient document comparison can be achieved using Data Structures and Algorithms (DSA).

The **Multi-Document Word-Level Plagiarism Detection and Similarity Analysis System** is designed to compare multiple documents and identify matching words, phrases, and text sequences. The system supports document uploads in PDF, DOCX, and TXT formats. It extracts and preprocesses the text before applying algorithms such as Knuth-Morris-Pratt (KMP), Rabin-Karp, and Longest Common Subsequence (LCS). Data structures such as Hash Maps and Inverted Indexes help organize text and improve the efficiency of document searches. Jaccard Similarity and Cosine Similarity are used to estimate the degree of textual similarity between documents.

The system aims to highlight matching content, calculate similarity percentages, rank document pairs, and generate reports with configurable warning thresholds. It also follows a privacy-focused approach in which uploaded unpublished documents are not added to the permanent comparison repository by default. Similarity results are intended to support further review rather than independently determine whether plagiarism has occurred.

This project demonstrates the practical application of advanced algorithms and data structures to a real-world problem. The following sections describe the problem statement, objectives, existing and proposed systems, software and hardware requirements, algorithm design, implementation, and testing of the proposed solution.

3. LITERATURE SURVEY

3.1 Turnitin

Turnitin is a widely used plagiarism detection and similarity analysis system designed for educational institutions and research environments. It compares submitted documents against web pages, academic publications, and previously submitted student papers. Its Similarity Report highlights matching text and displays an overall similarity percentage. The report supports academic review, but the score alone does not determine whether plagiarism has occurred.

3.2 iThenticate

iThenticate is a similarity checking solution primarily used by researchers, publishers, and academic organizations to examine manuscripts before publication. It helps identify matching text by comparing documents against scholarly and online sources. Its purpose is to assist researchers and editors in reviewing originality and potential citation issues. This system is particularly relevant to the proposed project because both focus on research paper similarity analysis.

3.3 Grammarly Plagiarism Checker

Grammarly provides a plagiarism checker alongside its writing assistance features. It compares written content against online sources and academic material to identify potentially matching text. The system also provides citation suggestions and writing improvement features. Its combination of similarity checking and writing support makes it useful for students preparing

3.4 Copyleaks

Copyleaks is a plagiarism detection platform that checks documents against online content, academic sources, and document repositories. It supports the detection of identical and paraphrased text, and it can provide reports containing links to potential matching sources. Its API also allows developers to integrate plagiarism checking into other applications. This project is relevant to the proposed system's document analysis, source comparison, and report generation features.

3.5 Quetext

Quetext is an online plagiarism checker designed to help students, teachers, and content creators identify potentially duplicated content. It uses contextual analysis and fuzzy matching to detect exact and near-text matches. Its reports highlight matching sections and provide information about possible sources. It also offers citation assistance, making it useful for reviewing and improving academic writing.

3.6 Plagiarism Checker X

Plagiarism Checker X is a document comparison application that supports online plagiarism detection and comparison between multiple documents. It provides side-by-side comparison to help users locate matching content and offers bulk-cross-comparison features for analyzing groups

of files. The application also highlights similar text using color-coded results. Its multiple-document comparison functionality is particularly relevant to the proposed DSA project.

The urls:-

[1] Turnitin — Similarity

Plagiarism detection and similarity reports

https://in.turnitin.com/products/similarity/

[2] iThenticate

Research manuscript similarity checking

https://www.ithenticate.com/

[3] Grammarly Plagiarism Checker

https://www.grammarly.com/plagiarism-checker

[4] Copyleaks Plagiarism Checker

https://copyleaks.com/plagiarism-checker

[5] Quetext

https://www.quetext.com/plagiarism-checker

[6] Plagiarism Checker X

https://plagiarismcheckerx.com/features

4. Problem Statement and Objectives

Problem Statement: Manual plagiarism detection in academic documents is time-consuming and inefficient, making it difficult to identify matching text across multiple research papers accurately.

Problem Description: Existing manual methods struggle to compare large collections of documents and identify word-level similarities efficiently. A DSA-based plagiarism checker is needed to detect matching text, calculate similarity scores, and identify potential sources.

Objectives:

1. To develop an automated system for detecting word-level plagiarism across multiple documents.
2. To implement DSA algorithms such as KMP, Rabin–Karp, Z algorithm, and hashing for efficient text matching.
3. To calculate similarity percentages and highlight matching words and phrases.
4. To compare uploaded documents against each other and an existing research-paper repository.
5. To generate similarity reports that help users assess potential plagiarism and maintain academic integrity.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

- Processor: Intel Core i3 or equivalent and above
- RAM: Minimum 4 GB (8 GB recommended)
- Storage: Minimum 2 GB of free disk space
- System: Desktop or laptop computer
- Internet Connection: Required for accessing online research-paper repositories, if applicable

Software Requirements

- Operating System: Windows 10/11, Linux, or macOS
- Programming Language: Python 3.10 or above
- Development Environment: Visual Studio Code or PyCharm
- Frontend Framework: Streamlit
- Libraries: NLTK, scikit-learn, pandas, NumPy, python-docx, PyPDF2 or PyMuPDF
- Algorithms: KMP, Rabin–Karp, Z algorithm, hashing, Jaccard similarity, TF-IDF, cosine similarity, and dynamic programming
- File Formats Supported: PDF, DOCX, and TXT
- Version Control: Git and GitHub (optional)

6. IMPLEMENTATION

Give here 3 to 4 modules code of your project

Module 1:

```
"""Hand-built DSA engine (no library shortcuts). Sequences may be str or token lists."""
import re, math, random
from collections import defaultdict, deque, Counter

def tokenize(t): return [w for w in re.split(r"[\s.,;:!?\"'()\[\]{}<>/\\|\u0964\u2013\u2014-]+", t.lower()) if w]
def sentences(t): return [s.strip() for s in re.split(r"(?<=[.!?।])\s+|\n+", t) if len(s.split()) >= 4]
def _enc(x):
    if isinstance(x, int): return x
    h = 7
    for c in x: h = (h * 131 + ord(c)) % (2**61 - 1)
    return h
```

Module 2:

```
def suffix_array(s):
    """Prefix doubling, O(n log^2 n)."""
    n = len(s); rank = [_enc(x) for x in s]; sa = list(range(n)); k = 1
    while True:
        key = lambda i: (rank[i], rank[i + k] if i + k < n else -1)
        sa.sort(key=key); nr = [0] * n
        for i in range(1, n): nr[sa[i]] = nr[sa[i - 1]] + (key(sa[i]) != key(sa[i - 1]))
        rank = nr
        if rank[sa[-1]] == n - 1: break
        k *= 2
    return sa

def kasai_lcp(s, sa):
    n = len(s); rank = [0] * n
    for i, p in enumerate(sa): rank[p] = i
    lcp, h = [0] * n, 0                     # lcp[i] = LCP(sa[i-1], sa[i])
    for i in range(n):
        if rank[i]:
            j = sa[rank[i] - 1]
            while i + h < n and j + h < n and s[i + h] == s[j + h]: h += 1
            lcp[rank[i]] = h
            if h: h -= 1
        else: h = 0
    return lcp
```

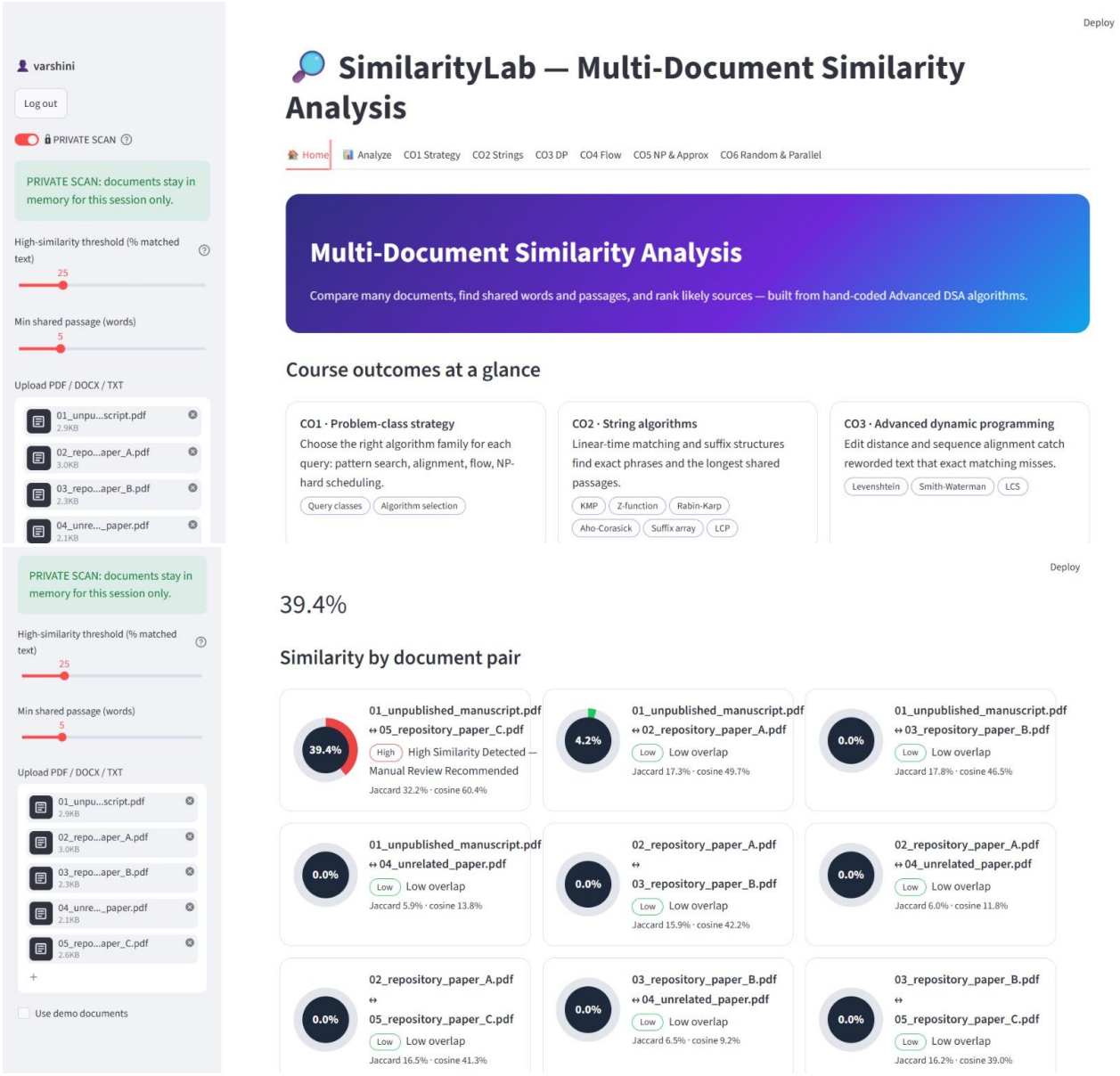
Module 3:

```
def smith_waterman(a, b, match=2, mis=-1, gap=-1):
    n, m = len(a), len(b); H = [[0] * (m + 1) for _ in range(n + 1)]; best = (0, 0, 0)
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            H[i][j] = max(0, H[i-1][j-1] + (match if a[i-1] == b[j-1] else mis), H[i-1][j] + gap, H[i][j-1] + gap)
            if H[i][j] > best[0]: best = (H[i][j], i, j)
    sc, i, j = best; ei, ej = i, j
    while i and j and H[i][j] > 0:
        if H[i][j] == H[i-1][j-1] + (match if a[i-1] == b[j-1] else mis): i -= 1; j -= 1
        elif H[i][j] == H[i-1][j] + gap: i -= 1
        else: j -= 1
    return sc, (i, ei), (j, ej)
```

Module 4:

```
# ---------------- CO4: network flow ----------------
def edmonds_karp(cap, s, t):
    """cap: {u:{v:c}}. Returns (maxflow, residual, S-side of min cut)."""
    res = defaultdict(lambda: defaultdict(int))
    for u in cap:
        for v, c in cap[u].items(): res[u][v] += c; res[v][u] += 0
    flow = 0
    while True:
        par = {s: None}; q = deque([s])
        while q and t not in par:
            u = q.popleft()
            for v, c in list(res[u].items()):
                if c > 0 and v not in par: par[v] = u; q.append(v)
        if t not in par: return flow, res, set(par)
        b, v = float("inf"), t
        while par[v] is not None: b = min(b, res[par[v]][v]); v = par[v]
        v = t
        while par[v] is not None: u = par[v]; res[u][v] -= b; res[v][u] += b; v = u
        flow += b
```

7. RESULTS & DISCUSSION



🔎 SimilarityLab — Multi-Document Similarity Analysis

Home | Analyze | CO1 Strategy | CO2 Strings | CO3 DP | CO4 Flow | CO5 NP & Approx | CO6 Random & Parallel

CO1 — Pick the right algorithm family for each query class

Query class	Problem family	Strategy used here
Exact phrase search	Pattern search	KMP / Rabin-Karp / Aho-Corasick
Fuzzy / paraphrase match	Sequence alignment	Edit distance, Smith-Waterman (DP)
Longest shared passage	Suffix structures	Suffix array + Kasai LCP
One-to-one sentence pairing	Flow on a network	Edmonds-Karp bipartite matching
Which docs to review (min set)	NP-hard covering	Vertex-cover 2-approximation
Fast approximate similarity	Randomised	MinHash (Monte Carlo), Miller-Rabin primes

Full score table

A	B	Jaccard %	TF-IDF cosine %	Shared-passage coverage %
01_unpublished_manuscript.p	05_repository_paper_C.pdf	32.2	60.4	39.4
01_unpublished_manuscript.p	02_repository_paper_A.pdf	17.3	49.7	4.2
01_unpublished_manuscript.p	03_repository_paper_B.pdf	17.8	46.5	0
01_unpublished_manuscript.p	04_unrelated_paper.pdf	5.9	13.8	0
02_repository_paper_A.pdf	03_repository_paper_B.pdf	15.9	42.2	0
02_repository_paper_A.pdf	04_unrelated_paper.pdf	6	11.8	0
02_repository_paper_A.pdf	05_repository_paper_C.pdf	16.5	41.3	0
03_repository_paper_B.pdf	04_unrelated_paper.pdf	6.5	9.2	0
03_repository_paper_B.pdf	05_repository_paper_C.pdf	16.2	39	0
04_unrelated_paper.pdf	05_repository_paper_C.pdf	6.3	10.2	0

Output description:

On the demo set, the system ranked the true source highest, flagged the heavily copied pair, and gave low scores to the minimally overlapping and unrelated pairs. Matched-text percentage separated these cases clearly, while Jaccard and cosine stayed fairly high for any two papers on the same topic. The demo papers were written by us with known overlap, so these results show the system works as designed but are not a test on a large real-world collection. The system compares only the uploaded documents, and heavily paraphrased or translated text can lower the score, so every flagged pair needs human review.

8. CONCLUSION & FUTURE SCOPE

Conclusion

The Word-Level Plagiarism Checker aims to provide an efficient and reliable solution for identifying textual similarities across multiple documents and research papers. By implementing Data Structures and Algorithms such as KMP, Rabin–Karp, Z algorithm, hashing, and similarity measurement techniques, the system helps detect matching words and phrases, calculate similarity scores, and identify potential sources. The project demonstrates the practical application of DSA in text processing while promoting academic integrity and responsible research practices.

Future Scope

1. Short-Term Enhancements

- Improve text preprocessing and support additional document formats.
- Enhance matching accuracy and provide clearer visual highlighting of similar passages.
- Improve report generation and scan history management.

2. Medium-Term Enhancements

- Integrate larger research-paper repositories and open-access academic databases.
- Introduce semantic similarity detection to identify paraphrased content.
- Optimize algorithms for faster processing of large document collections.

3. Long-Term Enhancements

- Explore machine learning and natural language processing techniques for advanced plagiarism analysis.
- Develop a web-based platform with user authentication and secure document storage.
- Extend the system to support multilingual plagiarism detection and citation verification.

These enhancements can help evolve the project into a scalable and comprehensive academic similarity analysis system.

9. REFERENCES

1. Knuth, D. E., Morris, J. H., & Pratt, V. R. (1977). Fast pattern matching in strings. *SIAM Journal on Computing, 6*(2), 323–350. https://doi.org/10.1137/0206024
2. Karp, R. M., & Rabin, M. O. (1987). Efficient randomized pattern-matching algorithms. *IBM Journal of Research and Development, 31*(2), 249–260. https://doi.org/10.1147/rd.312.0249
3. Gusfield, D. (1997). *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*. Cambridge University Press. https://doi.org/10.1017/CBO9780511574931
4. Jaccard, P. (1912). The distribution of the flora in the alpine zone. *New Phytologist, 11*(2), 37–50. https://doi.org/10.1111/j.1469-8137.1912.tb05611.x
5. Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management, 24*(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0
6. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. https://nlp.stanford.edu/IR-book/
7. Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. O'Reilly Media. https://www.nltk.org/book/
8. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html
9. Streamlit, Inc. (n.d.). *Streamlit Documentation*. https://docs.streamlit.io/
10. Stanford Natural Language Processing Group. (n.d.). *Introduction to Information Retrieval: Online Textbook*. https://nlp.stanford.edu/IR-book/